Using GPU to Run Monte-Carlo Simulation

Brook Hamilton

hamilbro@oregonstate.edu

CS 475: Parallel Programming

Project 5

Professor Jesse Rosenzweig

November 1, 2026

Oregon State University - Cascades

Introduction:

This project investigates the probability that the first snow of the year in Bend, Oregon, will fall on or before Thanksgiving. By collecting historical data of the earliest and latest snowfalls of the year, combined with the mean first snowfall and an estimated standard deviation, the Monte Carlo simulation was used to find the probability of snow on the target day. Box-Muller was used to transform the input of two standard, evenly distributed random numbers that are fed into the simulation to predict the probability of snow. Using OpenMP to create multiple threads, the simulations were run for up to a million different years to get an accurate probability of snow on Thanksgiving in Bend.

Unlike the first time we ran this experiment, we will now by using the GPU in order to divide up the work among arrays of threads on individual blocks within the GPU. This implementation allows for further parallelization as warps will be able to work while others must wait.

Machine Details:

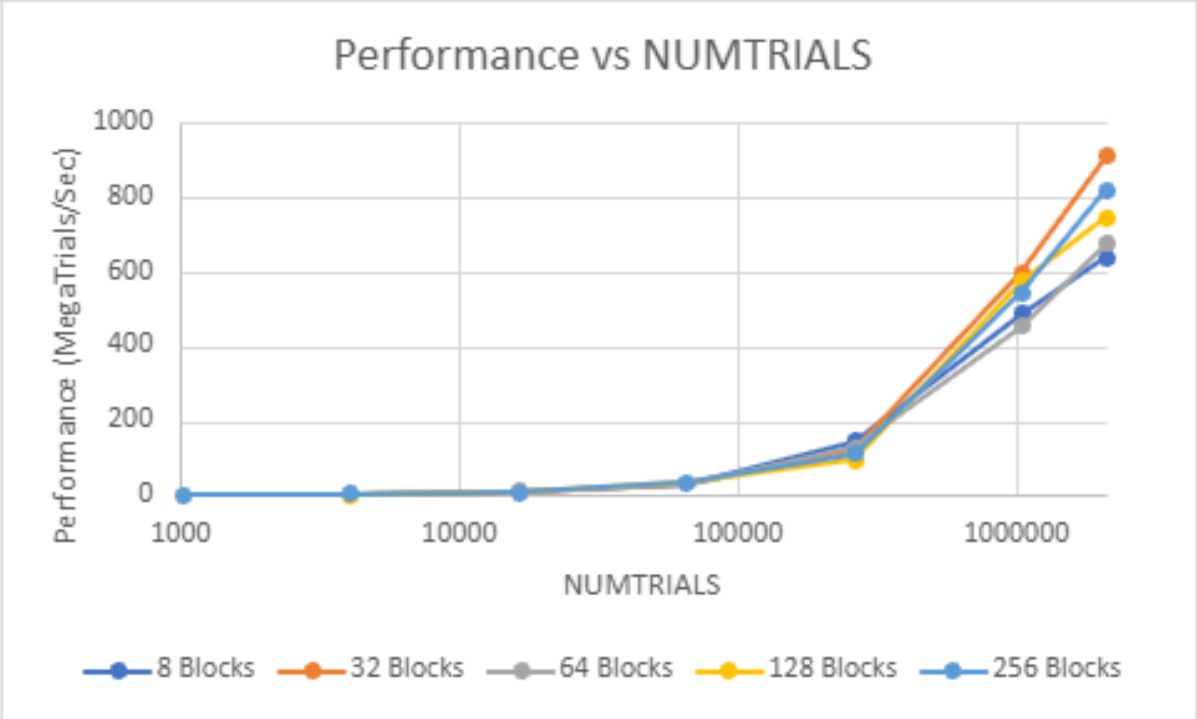
Machine: Lenovo LOQ AMD Ryzen 7 7435HS
Architecture: X86_64
CPU(s): 16
Thread(s) per core: 2
Core(s) per socket: 8
Operating System: Windows 11
Compiler: g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
GPU: NVIDIA GeForce RTX 4060 Laptop GPU

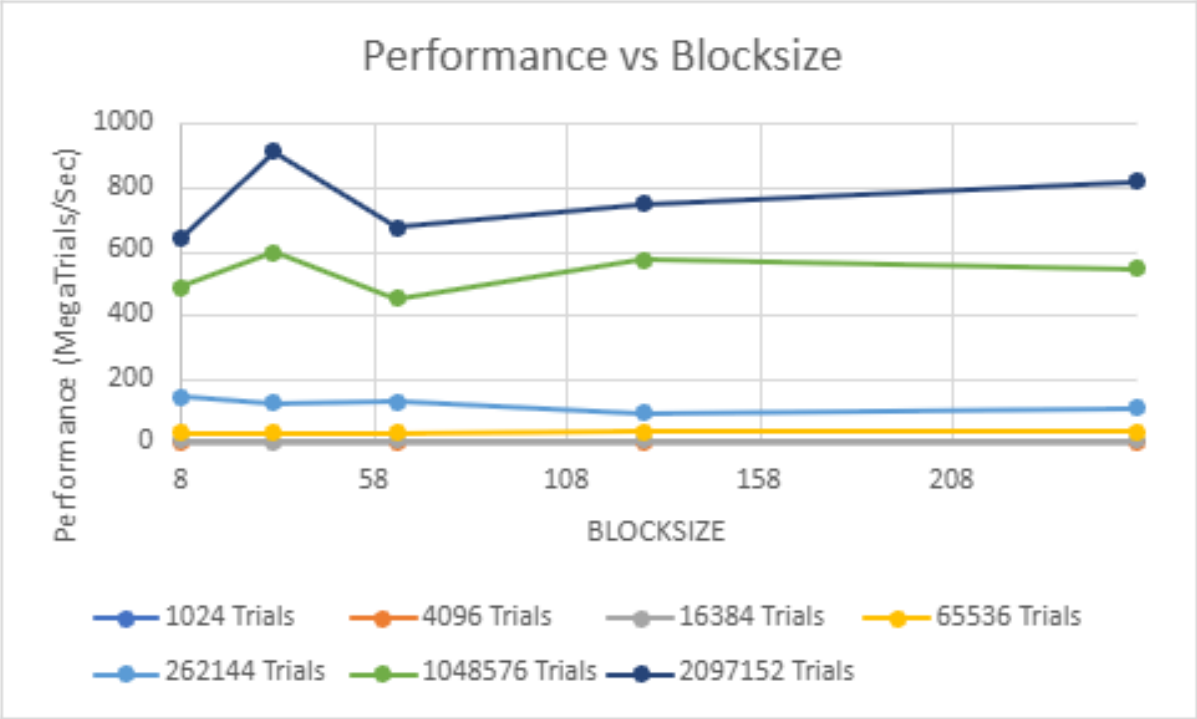
Results:

NUMTRIALS	BLOCKSIZE	MegaTrials/Sec	Probability
1024	8	0.387	88.96
1024	32	0.5118	87.5
1024	64	0.5971	88.77
1024	128	0.4008	88.48
1024	256	0.3968	88.57
4096	8	1.2595	87.04

4096	32	2.193	89.23
4096	64	2.082	88.33
4096	128	1.2755	87.92
4096	256	2.4375	88.53
16384	8	8.4122	88.31
16384	32	5.6477	88.54
16384	64	8.8938	88.02
16384	128	8.0808	88.07
16384	256	7.7973	88.22
65536	8	31.6944	88.34
65536	32	30.7632	88.23
65536	64	30.7494	88.23
65536	128	34.517	88.3
65536	256	34.0301	88.34
262144	8	145.5088	88.36
262144	32	123.7874	88.48
262144	64	129.485	88.36
262144	128	94.1133	88.38
262144	256	110.1994	88.4
1048576	8	488.5278	88.4
1048576	32	596.7909	88.42
1048576	64	453.2603	88.34
1048576	128	573.7699	88.43
1048576	256	544.337	88.38
2097152	8	639.5691	88.37

2097152	32	912.0336	88.37
2097152	64	674.3566	88.36
2097152	128	747.3771	88.37
2097152	256	819.0873	88.41





Probability of snow before Thanksgiving in bend: 88.38%

Conclusion:

The graphs generated from the results of the trials made some very interesting patterns. Probably most notably is that the performance rises very dramatically as the NUMTRIALS increase, rising from about 0.4 MegaTrials/sec at 1024 trials to 640–910 at 2,097,152. This pattern also shows that the climb in performance starts to level off between 1 million and 2 million trials, still performing very well but not as dramatic of a climb as shown from 1024-1 million. It does not seem that there is a clear winner when it comes to block size, block size of 8 is actually the best performer for 262,144 trials but the slowest for 2,097,152 trials.

The graphs look the way they do for many different reasons. At small trial counts, the overhead of launching the kernel takes most of the time, so it isn't efficient until there are many more trials. My results might look particularly different from other students because I ran it on my personal laptop (pelican was not working at the time i worked on this project) so my GPU was busy not only running this project but also my screen and other background applications as well. Since all the runs ran so fast it is possible that any noise from my computer, even a very small disruption changed the data a lot, even though the overall patterns are still visible. The leveling off of the GPU seen from 1M-2M trials is most likely due to the longer run times of more trials so the performance has a more difficult time rising as fast.

BLOCKSIZE 8 was expected to be the worst performing BLOCKSIZE because warps are 32 threads so 75% of the block was not working. But for me and what my data is displaying, this does not seem to be the case. My data shows that for 262,144 trials, block 8 was the fastest which seems very odd, but is most likely due to me running this on my personal laptop so there was a lot of other noise recorded in the run times.

Compared to project 1, this implementation is much better. By using the kernel on GPU to run the calculations, and the CPU to process the data, the program is able to run much faster than just using the CPU to do all of it. This comparison is very apparent when looking at the GPU's peak being much higher for larger trial counts than that in project 1 because the GPU is able to run thousands of threads at once instead of the CPU's 16 threads. That being said, because of how powerful the kernel is on larger trial counts, it also has some overhead to get it started up so the CPU only implementation is most likely still better for small trial counts.

This project showed a lot about what is both good and bad for GPU implementations of multithreading such as GPU's being very powerful with huge numbers of independent, and identical calculations such as the Monte Carlo simulation that we ran this time, pretty much a huge for-loop. It also showed that you need enough work for the GPU to do in order to make it worth it, which is why the CPU only implementation is better for smaller trial counts. With the performance problem that was supposed to have with a block size of 8, it also shows that you need to optimize and tune the problem to work for GPU computations and threading.